

MODULE 3: KNOWLEDGE & REASONING

Contents

1. Key Points.....	1
2. Steps of the Knowledge Engineering Process.....	2
3. Inference Engine	2
4. Forward Chaining (Data-Driven Reasoning).....	3
5. Backward Chaining (Goal-Driven Reasoning).....	4
6. What is Knowledge Representation?	7
7. Types of Knowledge.....	7
8. Knowledge Representation Challenges	7
9. Knowledge Representation Methods.....	7
10. Mapping Knowledge to Representation	8
11. Issues in Knowledge Representation.....	8
12. Inference Mechanisms.....	9

Key Points

1. **Logical Agents:** Agents that make decisions based on logical reasoning and knowledge representation. Use formal logic (like propositional or first-order logic) to derive conclusions from known facts and rules.
2. **Knowledge-Based Agents:** Agents that maintain a knowledge base of facts about the world and use it to make decisions. They apply logical inference rules to the knowledge base to decide actions.
3. **Semantics:** In logic, semantics refers to the meaning of symbols, sentences, or expressions in a formal system. It connects logical expressions to real-world situations, ensuring that the agent's inferences are meaningful.
4. **Forward and Backward Chaining:**
 - a. **Forward Chaining:** A method of reasoning that starts from known facts and applies inference rules to derive new facts (data-driven approach).
 - b. **Backward Chaining:** A goal-driven reasoning method that starts with a goal and works backward to find facts that support the goal.
5. **Agents Based on Propositional Logic:** Agents that use propositional logic to represent their knowledge base. Propositional logic deals with statements that are either true or false. **Example:** An agent uses a set of true/false statements about the world to make decisions.
6. **Hybrid Agent:** An agent that combines different approaches, such as logical reasoning and machine learning, to enhance decision-making. These agents use multiple techniques (logic-based reasoning, probability, learning, etc.) for more versatile and flexible behaviour.

7. Atomic and Complex Sentences:

- a. **Atomic Sentences:** Simple, indivisible statements in logic (e.g., "It is raining").
- b. **Complex Sentences:** Combinations of atomic sentences using logical connectives (e.g., "It is raining AND it is cold").

8. Term, Ground Term:

- a. **Term:** Represents objects in the domain of discourse in logic. Terms can be variables, constants, or functions.
- b. **Ground Term:** A term that contains no variables, only constants (e.g., "John" or "5").

9. First-Order Predicate Logic (FOPL): A more expressive form of logic than propositional logic. It includes quantifiers (e.g., $\forall$ for all, $\exists$ for some) and predicates that can take objects as arguments. **Key Feature:** Allows for expressing relationships between objects (e.g., "All humans are mortal").

Steps of the Knowledge Engineering Process

1. **Identify the Problem:** Define the specific problem the knowledge-based system will solve. Understand the domain and the scope of the problem.
2. **Knowledge Acquisition:** Collect and gather knowledge from experts, books, or data sources. Identify the key facts, rules, and procedures needed for decision-making.
3. **Knowledge Representation:** Choose a method for representing the acquired knowledge, such as:
 - Logic (propositional or first-order predicate logic).
 - Semantic networks.
 - Frames or rules.
4. **Design the Knowledge Base:** Organize the facts, rules, and knowledge into a structured format (e.g., knowledge base). Ensure it can be updated or modified as needed.
5. **Define Inference Mechanism:** Select an inference engine or reasoning technique (e.g., forward or backward chaining). Determine how the system will apply knowledge to solve problems.
6. **Develop the Knowledge-Based System:** Build the system, integrating the knowledge base and inference mechanism. Implement the system using AI programming languages (like Prolog or Python).
7. **Testing and Validation:** Test the system on sample problems or scenarios to ensure it behaves correctly. Validate its performance by comparing the system's output with expert solutions.
8. **Refinement:** Continuously refine the knowledge base and rules based on feedback and testing results. Address gaps or errors in the system's knowledge or reasoning.
9. **Deployment:** Deploy the system in the intended environment where users can interact with it.
10. **Maintenance and Updates:** Regularly update the knowledge base with new information. Adapt the system as the problem domain or requirements evolve.

Inference Engine

An **Inference Engine** is a component of a **knowledge-based system**, such as expert systems or artificial intelligence systems, that applies logical rules to a knowledge base (containing facts and rules) to deduce new information or solve problems. The inference engine processes the knowledge base and works through reasoning to arrive at conclusions. **Functionality:**

- It applies **rules of inference** to derive new facts from existing facts.

- The process of inferring new knowledge involves **applying rules** in a systematic way to the available data.
- It can handle both **forward chaining** and **backward chaining** as reasoning strategies.
- **Components:**
 - **Knowledge Base:** Contains facts and rules (in the form of IF-THEN statements).
 - **Inference Mechanism:** Applies logical reasoning techniques to draw conclusions from the knowledge base.
 - **Working Memory:** Stores intermediate facts during the reasoning process.

Forward Chaining (Data-Driven Reasoning)

Forward chaining starts with known facts and applies inference rules to extract new facts or conclusions. This process is repeated until the desired goal is reached or no more rules can be applied.

How it Works:

- Starts from the **initial facts** in the knowledge base.
- Uses **if-then rules** to infer new facts (if the **if** part of the rule is true, then the **then** part can be added to the knowledge base).
- Continues adding inferred facts to the knowledge base, progressing **forward**, until the goal or conclusion is reached.

Example (Rule-based system):

- **Rules:**
 - Rule 1: If it is raining, then the grass is wet.
 - Rule 2: If the grass is wet, then it is slippery outside.
- **Facts:** It is raining.
- **Process:**
 - From the fact "It is raining," Rule 1 is triggered, and the system concludes that "the grass is wet."
 - With the fact "the grass is wet," Rule 2 is triggered, leading to the conclusion that "it is slippery outside."

Applications:

- **Expert Systems:** Systems that diagnose diseases or recommend actions based on symptoms (e.g., medical diagnosis).
- **Production Systems:** Systems that make decisions in response to sensor inputs, like industrial robots or automated reasoning systems.

Advantages:

- **Data-Driven:** Starts from available data, making it suitable for situations where the system has access to many known facts.
- **Easy to Implement:** Straightforward logic flow, ideal for situations with many rules that generate many possible conclusions.

Disadvantages:

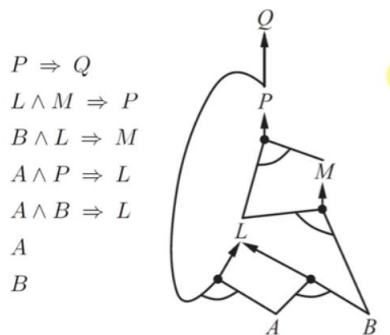
- **Inefficiency:** It may generate unnecessary conclusions that do not lead to the goal, especially if there are many irrelevant rules.

- **No Goal Focus:** It does not start with a goal, so it may explore paths that are not relevant to the problem.

Forward Chaining is useful when:

- You have many known facts but don't have a specific goal.
- You want to derive all possible conclusions from the facts.
- You are dealing with sensor-driven systems or production systems (e.g., automation, real-time systems).

Example



Backward Chaining (Goal-Driven Reasoning)

Backward chaining works by starting from the desired goal or conclusion and working **backward** to determine which facts must be true to support the goal. The process continues until the system reaches known facts or finds that the goal is unachievable.

How it Works:

- Starts from a **goal** (a fact or condition you want to prove or achieve).
- Searches for rules where the **then** part (conclusion) matches the goal.
- Checks whether the **if** part of that rule is true (i.e., tries to prove the conditions).
- If the conditions are not known, backward chaining attempts to prove them as sub-goals, working backward through the chain of logic.

Example (Rule-based system):

- **Goal:** Prove that "it is slippery outside."
- **Rules:**
 - Rule 1: If it is raining, then the grass is wet.
 - Rule 2: If the grass is wet, then it is slippery outside.
- **Process:**
 - The goal is "it is slippery outside," which matches Rule 2's conclusion.
 - To prove this, the system needs to check if the grass is wet (from the **if** part of Rule 2).
 - It now works backward to see if Rule 1's conditions hold (i.e., if "it is raining"), which is a known fact.
 - Once it verifies "it is raining," the system confirms the goal "it is slippery outside."

Applications:

- **Expert Systems:** Used in systems where a specific conclusion needs to be verified, such as legal reasoning or troubleshooting guides.

- **Prolog Programming:** Prolog, a logic programming language, uses backward chaining as its primary inference mechanism.

Advantages:

- **Goal-Driven:** More efficient when you have a specific goal, as it focuses on proving or disproving that goal.
- **Focused Search:** Avoids unnecessary exploration of irrelevant rules or facts, unlike forward chaining.

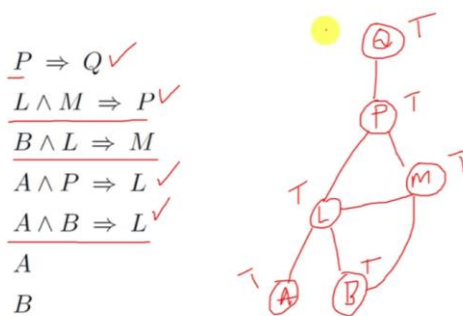
Disadvantages:

- **Limited Exploration:** Can miss relevant conclusions if the goal isn't properly defined.
- **Data Availability:** Needs to have rules that support working backward from the goal, and might struggle in domains with insufficient rules or facts.

Backward Chaining is useful when:

- You have a specific goal in mind and want to find out whether it can be achieved.
- You need to prove or disprove a hypothesis or diagnosis.
- You want to focus on a specific conclusion rather than explore all possible outcomes (e.g., Prolog systems, medical diagnosis).

Example



Feature	Forward Chaining	Backward Chaining
Type	Data-Driven (Starts from facts)	Goal-Driven (Starts from a goal)
Starting Point	Known facts in the knowledge base	A specific goal to be achieved
Direction	Works forward from facts to conclusions	Works backward from the goal to find supporting facts
Efficiency	May explore unnecessary rules or facts	More efficient when a specific goal is known
Use Cases	When many facts are available, and the goal isn't specified	When the goal is known and facts are limited
System Type	Used in expert systems and production systems	Used in diagnostic systems and legal reasoning
Computational Complexity	May be computationally expensive due to the wide exploration of facts	More efficient in smaller problem spaces with focused goals

Example

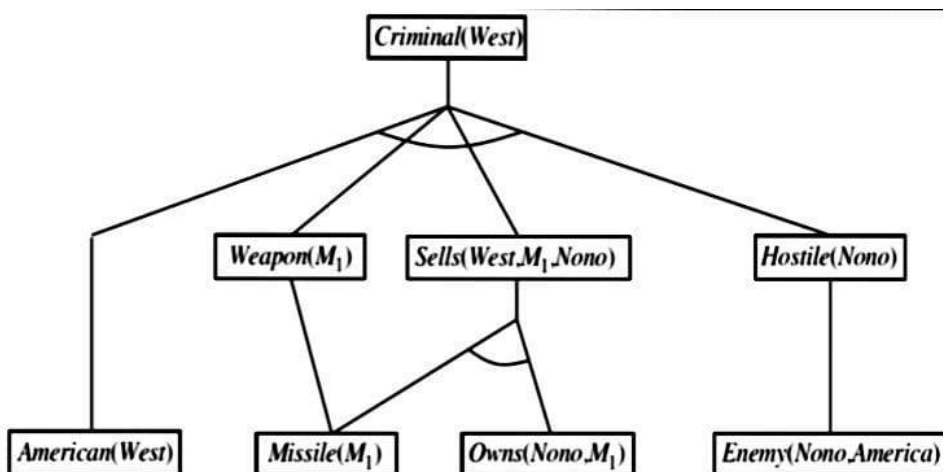
The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American. An enemy of America counts as "hostile".

Prove that: West is a Criminal using i) Forward chaining ii) Backward chaining

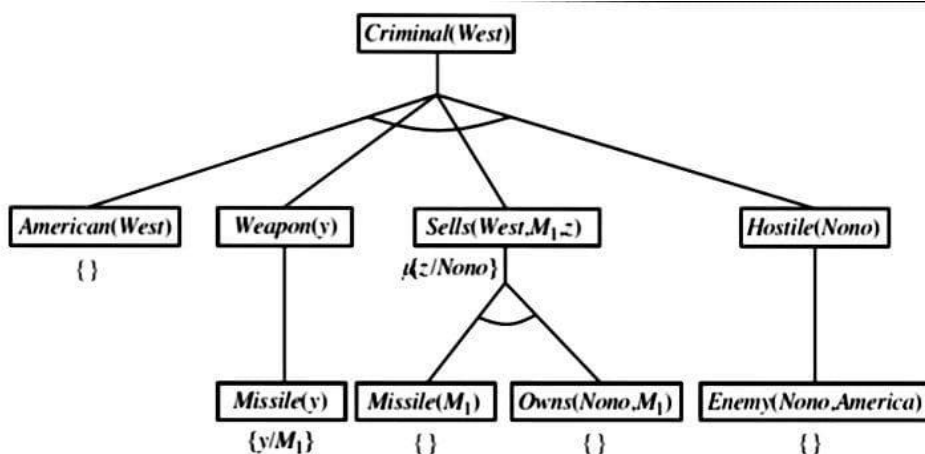
Solution:

1. $\forall x, y, z \text{ American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$
2. $\forall x \text{ Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$
3. $\forall x \text{ Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$
4. $\forall x \text{ Missile}(x) \Rightarrow \text{Weapon}(x)$
5. $\text{American}(\text{West})$
6. $\text{Enemy}(\text{Nono}, \text{America})$
7. $\text{Owns}(\text{Nono}, M1)$ and
8. $\text{Missile}(M1)$

Forward Chaining:



Backword Chaining:



What is Knowledge Representation?

Knowledge Representation (KR) is the field of AI concerned with how knowledge about the world can be formally represented and structured in a way that a computer can process, reason, and make decisions.

KR allows machines to store, retrieve, and manipulate data to simulate human-like understanding and reasoning.

- **Role in AI:** KR plays a central role in AI because it forms the foundation for tasks like problem-solving, learning, decision-making, and natural language processing. The way knowledge is represented impacts the performance and efficiency of AI systems.
- **Importance in Reasoning and Decision-Making:**
 - It allows AI systems to **reason** about the world by drawing inferences from the represented knowledge.
 - It is crucial for decision-making processes, enabling the system to select optimal actions based on the available facts, constraints, and goals.

Types of Knowledge

1. **Declarative Knowledge:** Represents **facts** about the world (e.g., "Paris is the capital of France"). **Static knowledge:** Can be directly stated or queried without needing action to be performed (e.g., facts about objects or entities). Example: "John is a student."
2. **Procedural Knowledge:** Represents the knowledge of **how to do things** or the steps required to perform a task (e.g., how to cook a dish or solve a problem). **Dynamic knowledge:** Often involves algorithms, procedures, or workflows. Example: The steps to bake a cake.
3. **Factual vs. Inferential Knowledge:** **Factual Knowledge:** Represents **what** is true, raw information (e.g., "The sky is blue"). Example: "It is raining."
4. **Inferential Knowledge:** Knowledge derived from facts or rules, **how** things are related (e.g., "If it rains, the ground will be wet"). Example: "The ground is wet."

Knowledge Representation Challenges

1. **Expressiveness:** The system's ability to represent diverse real-world knowledge, including relationships, hierarchies, and constraints.
2. **Efficiency:** The system's capability to quickly access and manipulate knowledge for fast inference and decision-making.
3. **Scalability:** The system's ability to handle large knowledge bases without losing performance or accuracy as new facts or rules are added.
4. **Interpretability:** The ease with which humans can understand and interpret the system's knowledge and outputs.

Knowledge Representation Methods

1. **Semantic Networks:**
 - a. A graphical representation where **nodes represent concepts** or entities, and **edges represent relationships** between them.
 - b. Useful for representing **hierarchies** (e.g., "A dog is an animal").
 - c. Example: **Node:** "Dog", **Node:** "Animal", **Edge:** "is a"
2. **Frames:**

- a. Data structures used to represent **stereotyped situations**, often containing slots (attributes) and fillers (values).
- b. Frames are like objects in object-oriented programming, where specific data is stored in predefined attributes.
- c. Example: A frame for a "Person" could have attributes like "Name," "Age," and "Gender."
- 3. **Logic (Propositional, Predicate Logic):**
 - a. **Propositional Logic:** Deals with simple statements that can be true or false (e.g., "The sky is blue").
 - b. **Predicate Logic:** Allows representation of more complex statements involving **quantifiers** (like "For all" or "There exists") and predicates (e.g., "Person(John)").
 - c. Predicate logic allows expressing facts like "John is a student" and applying rules to infer new facts.
- 4. **Rules (IF-THEN):**
 - a. **Rule-based representation** where facts or conditions (IF) lead to actions or conclusions (THEN).
 - b. Example: "IF it is raining THEN the ground is wet."
 - c. Commonly used in **expert systems**.

Mapping Knowledge to Representation

- 1. **Formalization:**
 - a. The process of converting human-readable knowledge into a **formal system** that a computer can process.
 - b. Formalizing knowledge ensures that it can be reasoned about by machines in a consistent and logical way.
- 2. **Ontologies:**
 - a. A formal specification of a **conceptualization** (i.e., a shared understanding of a domain).
 - b. Ontologies represent a set of **concepts** and the **relationships** between them, allowing for structured and explicit knowledge representation.
 - c. Example: An ontology for an e-commerce system might include concepts like "Product", "Customer", and "Order".
- 3. **Taxonomies:**
 - a. Hierarchical classification of concepts or objects in a domain.
 - b. Used to categorize entities into **parent-child** relationships (e.g., "Mammals" is a parent to "Dogs" and "Cats").
- 4. **Knowledge Graphs:**
 - a. A graph-based structure that represents **entities** as nodes and **relationships** between them as edges.
 - b. Knowledge graphs integrate various types of knowledge and allow for **reasoning** based on connected nodes (e.g., Google Knowledge Graph).

Issues in Knowledge Representation

- 1. **Ambiguity and Vagueness**
 - a. **Handling Imprecise Information (Fuzzy Logic):**
 - i. Fuzzy logic is a system that allows for **gradual truth values** rather than the binary true/false used in classical logic.
 - ii. It handles uncertainty and vagueness, for example, "tall" can be represented as a range instead of a precise value.
 - b. **Example of Ambiguous Terms:**

- i. Words like “**tall**”, “**soon**”, or “**hot**” have fuzzy boundaries and can vary depending on context. Fuzzy logic provides a way to represent and reason with such imprecise terms.

2. Incompleteness

a. **Dealing with Incomplete Knowledge (Assumption-based Reasoning):**

- i. In many situations, knowledge is incomplete, and assumptions are made to proceed with reasoning.
- ii. **Default reasoning:** Using defaults when information is missing (e.g., "If the person's age is not specified, assume the person is an adult").

3. Consistency

- a. **Maintaining Consistency in the Knowledge Base:** Knowledge systems must ensure that the facts stored do not contradict each other. Inconsistent knowledge leads to incorrect reasoning or conclusions.
- b. **Conflict Resolution:** Mechanisms to resolve conflicts in knowledge bases, such as prioritizing rules or using conflict resolution strategies (e.g., **prefer the most recent fact**).

4. Modularity

- a. **Importance of Modularity:** Knowledge systems must support **modularity** so that new knowledge can be added, updated, or removed without affecting other parts of the system.
- b. **Benefits for Adding/Removing Knowledge:** Modularity allows for **easy updates** and **maintenance**, as components of the knowledge base can be modified independently.

Inference Mechanisms

1. **Deductive Reasoning:** The process of drawing specific conclusions from general premises or facts. In deductive reasoning, if the premises are true, the conclusion must also be true. **Used in:** Logical proofs, formal systems, and rule-based systems. **Example:**
 - a. Premise 1: All humans are mortal.
 - b. Premise 2: Socrates is a human.
 - c. Conclusion: Socrates is mortal.
2. **Inductive Reasoning:** The process of generalizing from specific observations or examples to form a general rule or conclusion. The conclusions in inductive reasoning are probable, not certain. **Used in:** Machine learning, pattern recognition, and statistical reasoning. **Example:**
 - a. Observation: The sun has risen every day in the past.
 - b. Conclusion: The sun will rise tomorrow.
3. **Non-monotonic Reasoning:** In non-monotonic reasoning, conclusions drawn from a set of facts can be **revised** if new information is added, unlike in deductive reasoning where conclusions remain true regardless of additional information. **Used in:** Reasoning with incomplete, evolving, or contradictory knowledge (e.g., legal or medical reasoning). **Example:**
 - a. Fact 1: All birds can fly.
 - b. Fact 2: A penguin is a bird.
 - c. Conclusion: A penguin can fly.
 - d. **New Fact:** Penguins cannot fly.
 - e. Revised Conclusion: Penguins cannot fly.